

# Fractions

Nous nous proposons d'écrire une classe `Fraction` permettant de créer et manipuler des fractions. Une fraction est définie par 2 entiers,  $n$  son numérateur et  $d$  son dénominateur et est habituellement notée sur une ligne :  $n/d$ . Le dénominateur d'une fraction doit toujours être différent de 0.

Une fraction est dite « réduite » lorsque son numérateur et son dénominateur sont premiers entre eux (par exemple  $6/15$  n'est pas réduite, alors que  $2/5$  est réduite et équivalente). Pour « réduire » une fraction, il suffit de diviser son numérateur et son dénominateur par le plus grand commun diviseur (ou *pgcd*) de son numérateur et de son dénominateur. Les instances de la classe définie dans le cadre de ce TM devront toutes représenter des fractions réduites.

On rappelle ci-dessous comment calculer le résultat d'opérations arithmétiques simples sur des fractions.

- **Addition** :  $n_1/d_1 + n_2/d_2 = n/d$ , où  $n = n_1 * p/d_1 + n_2 * p/d_2$  et  $d = p$ , avec  $p$  le *ppcm* de  $d_1$  et  $d_2$ ,
- **Soustraction** :  $n_1/d_1 - n_2/d_2 = n/d$ , où  $n = n_1 * p/d_1 - n_2 * p/d_2$  et  $d = p$ , avec  $p$  le *ppcm* de  $d_1$  et  $d_2$ ,
- **Multiplication** :  $n_1/d_1 * n_2/d_2 = n/d$ , où  $n = n_1 * n_2$  et  $d = d_1 * d_2$ ,
- **Division** :  $n_1/d_1 : n_2/d_2 = n/d$ , où  $n = n_1 * d_2$  et  $d = d_1 * n_2$  (remarque : on note dans ce TM la division de fractions à l'aide du caractère `:` pour éviter toute confusion avec `/` séparant le numérateur et le dénominateur).

On rappelle également que le *ppcm* de 2 entiers  $a$  et  $b$  peut être obtenu grâce à la formule  $\text{ppcm}(a, b) = a / \text{pgcd}(a, b) * b$  et que pour calculer le *pgcd* de 2 entiers  $a$  et  $b$ , on peut utiliser l'algorithme d'Euclide :

1. on calcule  $r$ , le reste de la division entière de  $a$  par  $b$ ,
2. si  $r$  est 0, alors le *pgcd* ( $a, b$ ) est  $b$ ,
3. si  $r$  est différent de 0 alors le *pgcd* ( $a, b$ ) est égal au *pgcd* ( $b, r$ ).

## Question 1

Définir la classe `Fraction` dotée des constructeurs suivants :

|                                            |                                                                             |
|--------------------------------------------|-----------------------------------------------------------------------------|
| <code>public Fraction()</code>             | Permet d'obtenir la fraction 0/1.                                           |
| <code>public Fraction(int n)</code>        | Permet d'obtenir la fraction n/1.                                           |
| <code>public Fraction(int n, int d)</code> | Permet d'obtenir la fraction n/d.                                           |
| <code>public Fraction(String ch)</code>    | Permet d'obtenir une fraction d'après son expression dans <code>ch</code> . |

Définir pour la classe `Fraction` les méthodes suivantes :

|                                                        |                                                                                                                                                                                                                           |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public int getNumérateur()</code>                | Accesseur en lecture pour le numérateur.                                                                                                                                                                                  |
| <code>public int getDénominateur()</code>              | Accesseur en lecture pour le dénominateur.                                                                                                                                                                                |
| <code>public void setNumérateur(int n)</code>          | Accesseur en écriture pour le numérateur.                                                                                                                                                                                 |
| <code>public void setDénominateur(int d)</code>        | Accesseur en écriture pour le dénominateur.                                                                                                                                                                               |
| <code>public String toString()</code>                  | Retourne une <code>String</code> contenant l'expression de <b>this</b> .                                                                                                                                                  |
| <code>public static Fraction valueOf(String ch)</code> | Retourne une <code>Fraction</code> dont l'expression est dans <code>ch</code> , <code>ch</code> est de la forme : expression d'un entier positif, suivie de <code>'/'</code> , suivi de l'expression d'un entier positif. |

|                                                         |                                                                                                                                                                                                                     |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>public int<br/>compareTo(Fraction f)</code>       | Retourne un <code>int</code> négatif si <code>this</code> est inférieur à <code>f</code> , positif si <code>this</code> est supérieur à <code>f</code> , nul si <code>this</code> est équivalent à <code>f</code> . |
| <code>public boolean equals(Object<br/>obj)</code>      | Retourne <code>true</code> si et seulement si <code>this</code> est équivalent à <code>obj</code> .                                                                                                                 |
| <code>public Fraction<br/>plus(Fraction f)</code>       | Retourne le résultat de l'addition de <code>this</code> et <code>f</code> .                                                                                                                                         |
| <code>public Fraction<br/>moins(Fraction f)</code>      | Retourne le résultat de la soustraction de <code>this</code> et <code>f</code> .                                                                                                                                    |
| <code>public Fraction<br/>fois(Fraction f)</code>       | Retourne le résultat de la multiplication de <code>this</code> par <code>f</code> .                                                                                                                                 |
| <code>public Fraction<br/>diviseePar(Fraction f)</code> | Retourne le résultat de la division de <code>this</code> par <code>f</code> .                                                                                                                                       |

## Question 2

Écrire un programme (cf Illustration) qui calcule des expressions arithmétiques de fractions toutes strictement positives, comme par exemple :

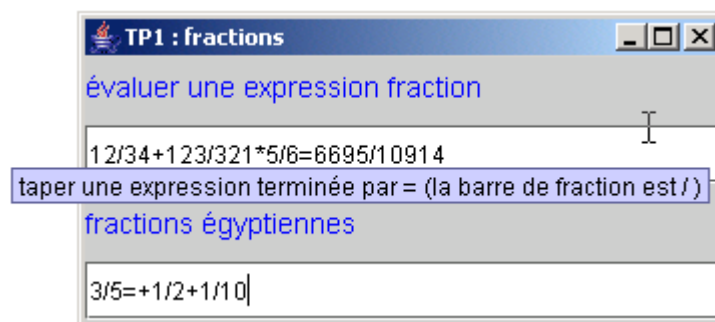
$$12/34+123/321*5/6=6695/10914$$

La première partie, jusqu'à '=' est tapée par l'utilisateur du programme, puis après la frappe de '=' le programme affiche le résultat.

Pour écrire ce programme, on fait les hypothèses suivantes :

1. Il n'y a jamais d'erreur dans les données tapées par l'utilisateur du programme.
2. Les opérateurs ont la même priorité, et l'opérateur le plus à gauche est évalué en premier. L'expression précédente correspond donc au parenthésage :

$$(12/34 + 123/321) * 5/6$$



## Question 3

Écrire un programme (cf Illustration) qui traduit une fraction quelconque en sa « notation égyptienne » : toute fraction est notée sous la forme  $1/d_1 + 1/d_2 + 1/d_3 + \dots + 1/d_n$ . Cette décomposition n'est pas unique, la méthode est donc au choix du programmeur.  
Exemple :  $3/5 = 1/2 + 1/10$